```
import pandas as pd

data = pd.read_csv("data/songs_train.csv")
test_data = pd.read_csv("data/songs_test.csv")

#make pandas show all columns
pd.set_option('display.max_columns', None)
pd.set_option('display.max_rows', None)
data.head()
```

	id	track_id	track_name	track_artist	track_popularity	track_album_id	track_album_name	track_albu
0	0	7v71Aur70pf46NQIoSQii7	Knock You Out - Hardwell Remix	Bingo Players	0	7q5EDnr3FhMjwSMQciMqqH	Knock You Out (Remixes)	
1	1	1oTHteQbmJw15rPxPVXUTv	Insane in the Brain	Cypress Hill	70	02lktkm4J7K7N8T63Gm7KX	Black Sunday	
2	2	44evOHkEKmJ5VofpPo0ta9	Chill Cloud	Joan Ember	39	7Bopcb1Ld22bhXdzddvwot	Chill Cloud	
3	3	2b6i9zx3ULzKrNWmyP0ePD	C90 (Remix)	John C	78	16a33enqgAGsnuMmRH7uHa	C90 (Remix)	
4	4	6DrUHF3EWygcmQ0DIF8KP4	Te Olvidaré	MYA	67	4WTOdIQXRzSV6xCZbfKJna	Te Olvidaré	

```
data.shape
```

```
(26266, 24)
```

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 26266 entries, 0 to 26265
Data columns (total 24 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   id                        26266 non-null  int64
 1   track_id                  26266 non-null  object
 2   track_name                26262 non-null  object
 3   track_artist              26262 non-null  object
 4   track_popularity          26266 non-null  int64
 5   track_album_id            26266 non-null  object
 6   track_album_name          26262 non-null  object
 7   track_album_release_date  26266 non-null  object
 8   playlist_name             26266 non-null  object
 9   playlist_id               26266 non-null  object
 10  playlist_genre            26266 non-null  object
 11  playlist_subgenre         26266 non-null  object
 12  danceability              26266 non-null  float64
 13  energy                    26266 non-null  float64
 14  key                       26266 non-null  int64
 15  loudness                  26266 non-null  float64
 16  mode                      26266 non-null  int64
 17  speechiness               26266 non-null  float64
 18  acousticness              26266 non-null  float64
 19  instrumentalness          26266 non-null  float64
 20  liveness                  26266 non-null  float64
 21  valence                   26266 non-null  float64
 22  tempo                     26266 non-null  float64
 23  duration_ms               26266 non-null  int64
dtypes: float64(9), int64(5), object(10)
memory usage: 4.8+ MB
```

```
data.duplicated().sum()
```

```
np.int64(0)
```

```
data.nunique()
```

```
id                        26266
track_id                  23205
```

```
track_name                19601
track_artist               9417
track_popularity            101
track_album_id            18896
track_album_name          16755
track_album_release_date   4165
playlist_name               449
playlist_id                 471
playlist_genre                6
playlist_subgenre            24
danceability                813
energy                      941
key                          12
loudness                   9525
mode                          2
speechiness                1252
acousticness               3572
instrumentalness           4514
liveness                   1582
valence                    1309
tempo                     15205
duration_ms               16967
dtype: int64
```

```
data.isnull().sum()
```

```
id                        0
track_id                  0
track_name                4
track_artist              4
track_popularity          0
track_album_id            0
track_album_name          4
track_album_release_date  0
playlist_name             0
playlist_id               0
playlist_genre            0
playlist_subgenre         0
danceability              0
energy                    0
key                       0
loudness                  0
mode                      0
speechiness               0
acousticness              0
instrumentalness          0
liveness                  0
valence                   0
tempo                     0
duration_ms               0
dtype: int64
```

```
data.dropna(inplace=True)
data.shape
```

```
(26262, 24)
```

```
data['track_album_release_date'] = pd.to_datetime(data['track_album_release_date'], format='mixed' )
test_data['track_album_release_date'] = pd.to_datetime(test_data['track_album_release_date'], format='mixed')
```

```
min_date = data['track_album_release_date'].min()
max_date = data['track_album_release_date'].max()
data['track_album_release_date_normalized'] = (data['track_album_release_date'] - min_date) / (max_date - min_date)
data.drop('track_album_release_date', axis=1, inplace=True)

test_data['track_album_release_date_normalized'] = (test_data['track_album_release_date'] - min_date) / (max_date - min_date)
test_data.drop('track_album_release_date', axis=1, inplace=True)
```

drop some features because data is very big

```
test_id = test_data['id']
```

```
data.drop(['id','track_id', 'track_name', 'track_album_name', 'track_album_id', 'track_artist',  'playlist_id'], axis=1, inplace=Tr
test_data.drop(['id','track_id', 'track_name', 'track_album_name', 'track_album_id', 'track_artist', 'playlist_id'], axis=1, inplac
```

```
test_data.shape
```

```
(6567, 16)
```

```
# strip and lower string columns to ensure uniformity before one hot encoding
from sklearn.preprocessing import OneHotEncoder

def clean_string_columns_encode(data, test_data):
    for col in data.select_dtypes(include=['object']).columns:
        data[col] = data[col].str.strip().str.lower()
        test_data[col] = test_data[col].str.strip().str.lower()
    encoder = OneHotEncoder(sparse_output=False, handle_unknown='ignore')
    encoded_data = encoder.fit_transform(data.select_dtypes(include=['object']))
    encoded_test_data = encoder.transform(test_data.select_dtypes(include=['object']))
    encoded_cols = encoder.get_feature_names_out(data.select_dtypes(include=['object']).columns)
    encoded_df = pd.DataFrame(encoded_data, columns=encoded_cols, index=data.index)
    encoded_test_df = pd.DataFrame(encoded_test_data, columns=encoded_cols, index=test_data.index)
    data = pd.concat([data.drop(columns=data.select_dtypes(include=['object']).columns), encoded_df], axis=1)
    test_data = pd.concat([test_data.drop(columns=test_data.select_dtypes(include=['object']).columns), encoded_test_df], axis=1)

    return data, test_data

data, test_data = clean_string_columns_encode(data, test_data)

data.shape
```

```
(26262, 477)
```

```
test_data.shape
```

```
(6567, 476)
```

```
from sklearn.model_selection import train_test_split
X = data.drop('track_popularity', axis=1)
y = data['track_popularity']
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.1, random_state=42)
```

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_val = scaler.transform(X_val)
X_test = scaler.transform(test_data.values)
```

```
c:\Users\ahmed\anaconda3\envs\torch-gpu\lib\site-packages\sklearn\utils\validation.py:2749: UserWarning: X does not have valid featu
  warnings.warn(
```

Neural Network model with py torch

```
import torch
from torch import nn
from torch.utils.data import DataLoader

batch_size = 256

X_train_tensor = torch.tensor(X_train, dtype=torch.float32)
y_train_tensor = torch.tensor(y_train.values, dtype=torch.float32).unsqueeze(1)
train_dataset = torch.utils.data.TensorDataset(X_train_tensor, y_train_tensor)
X_val_tensor = torch.tensor(X_val, dtype=torch.float32)
y_val_tensor = torch.tensor(y_val.values, dtype=torch.float32).unsqueeze(1)
val_dataset = torch.utils.data.TensorDataset(X_val_tensor, y_val_tensor)
train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)


X_test = torch.tensor(X_test, dtype=torch.float32)
test_dataset = torch.utils.data.TensorDataset(X_test)
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)
```

```
C:\Users\ahmed\AppData\Local\Temp\ipykernel_31476\342695387.py:17: UserWarning: To copy construct from a tensor, it is recommended t
  X_test = torch.tensor(X_test, dtype=torch.float32)
```

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

class NeuralNetwork(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Linear(X_train.shape[1], 128),
            # nn.BatchNorm1d(128),
            nn.ReLU(),
            nn.Dropout(0.2),

            nn.Linear(128, 64),
            # nn.BatchNorm1d(64),
            nn.ReLU(),
            nn.Dropout(0,2),

            # nn.Linear(64, 32),
            # # nn.BatchNorm1d(32),
            # nn.ReLU(),
            # nn.Dropout(0.1),
            nn.Linear(64, 1),
        )

    def forward(self, x):
        return self.layers(x)

model = NeuralNetwork().to(device)
loss_fn = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.0006)
```

```
def train(dataloader, model, loss_fn, optimizer):
    size = len(dataloader.dataset)
    model.train()
    for batch, (X, y) in enumerate(dataloader):
        X, y = X.to(device), y.to(device)

        # Compute prediction error
        pred = model(X)
        loss = loss_fn(pred, y) # This works directly now

        # Backpropagation
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()

        if batch % 100 == 0:
            loss_item = loss.item()
            current = (batch + 1) * len(X)
            # Print RMSE (sqrt of MSE) as it's more interpretable (on the same scale as popularity)
            print(f"MSE loss: {loss_item:>7f}  [{current:>5d}/{size:>5d}]")
            print(f"RMSE loss: {torch.sqrt(loss).item():>7f}  [{current:>5d}/{size:>5d}]")
```

```
def test(dataloader, model, loss_fn):
    num_batches = len(dataloader)
    model.eval()
    test_loss = 0
    with torch.no_grad():
        for X, y in dataloader:
            X, y = X.to(device), y.to(device)
            pred = model(X)
            test_loss += loss_fn(pred, y).item()

    test_loss /= num_batches
    test_rmse = torch.sqrt(torch.tensor(test_loss)) # Convert to RMSE
    print(f"Validation MSE: \n Avg MSE: {test_loss:>8f} \n")
    print(f"Validation RMSE: \n Avg RMSE: {test_rmse:>8f} \n")
    return test_loss # Return loss for early stopping
```

```
epochs = 100
best_val_loss = float('inf') # Keep track of the best loss
epochs_no_improve = 0
patience = 10 # Stop after 10 epochs of no improvement
```

```
for t in range(epochs):
    print(f"Epoch {t+1}\n-------------------------------")
    train(train_loader, model, loss_fn, optimizer)
    val_loss = test(val_loader, model, loss_fn)

    if val_loss < best_val_loss:
        best_val_loss = val_loss
        epochs_no_improve = 0
        # Save your best model
        torch.save(model.state_dict(), 'best_model.pth')
        print("Validation loss improved, saving model.")
    else:
        epochs_no_improve += 1
        print(f"Validation loss did not improve. Count: {epochs_no_improve}")

    if epochs_no_improve >= patience:
        print(f"Early stopping triggered after {t+1} epochs.")
        break

print("Done!")

# Load the best model weights before making predictions
model.load_state_dict(torch.load('best_model.pth'))
```

```
Epoch 1
-------------------------------
MSE loss: 2412.189453  [  256/23635]
RMSE loss: 49.114044  [  256/23635]
Validation MSE:
 Avg MSE: 856.519021

Validation RMSE:
 Avg RMSE: 29.266346

Validation loss improved, saving model.
Epoch 2
-------------------------------
MSE loss: 826.546021  [  256/23635]
RMSE loss: 28.749714  [  256/23635]
Validation MSE:
 Avg MSE: 410.371482

Validation RMSE:
 Avg RMSE: 20.257627

Validation loss improved, saving model.
Epoch 3
-------------------------------
MSE loss: 444.002380  [  256/23635]
RMSE loss: 21.071363  [  256/23635]
Validation MSE:
 Avg MSE: 407.702576

Validation RMSE:
 Avg RMSE: 20.191647

Validation loss improved, saving model.
Epoch 4
-------------------------------
MSE loss: 379.377228  [  256/23635]
RMSE loss: 19.477608  [  256/23635]
Validation MSE:
 Avg MSE: 404.478000

Validation RMSE:
 Avg RMSE: 20.111639

Validation loss improved, saving model.
Epoch 5
-------------------------------
MSE loss: 365.492920  [  256/23635]
RMSE loss: 19.117868  [  256/23635]
Validation MSE:
 Avg MSE: 404.149802

Validation RMSE:
 Avg RMSE: 20.103477

Validation loss improved, saving model.
Epoch 6
-------------------------------
```

```
predictions = []
model.eval()
with torch.no_grad():
    for X in test_loader:
        X = X[0].to(device) # X[0] because test_loader only has X, not (X, y)

        pred = model(X)

        # Squeeze to remove the [batch_size, 1] dim and make it [batch_size]
        # Round to the nearest integer
        # Clamp values to be between 0 and 100
        # Move to CPU and numpy
        predicted_values = torch.clamp(torch.round(pred.squeeze()), 0, 100).cpu().numpy()

        predictions.extend(predicted_values)

# Create submission
submission_df = pd.DataFrame({'id': test_id, 'track_popularity': predictions})
# Ensure predictions are integers for submission
submission_df['track_popularity'] = submission_df['track_popularity'].astype(int)
submission_df.to_csv('submission.csv', index=False)

print("Submission file created!")
```

```
Submission file created!
```